The String Class in Java

The `String` class in Java is part of the `java.lang` package and is used to represent a sequence of characters. Strings in Java are **immutable**, meaning once a string object is created, its value cannot be changed. Any operation that modifies a string actually creates a new string object.

Key Characteristics of the `String` Class:

1. **Immutable**: Once a `String` object is created, it cannot be modified.
2. **Object in Nature**: A `String` is an object, not a primitive data type.
3. **String Pool**: Strings are stored in a special memory region called the **string pool** to save memory by reusing common strings.

Commonly Used Methods in the `String` Class

The `String` class provides a wide range of methods for manipulating and processing strings. Below are some of the most commonly used methods:

1. `length()`

- Returns the length of the string (number of characters).
- **Syntax**:

```
int length();
```

- **Example**:

```
String str = "Hello";
System.out.println(str.length()); // Output: 5
```

2. `charAt(int index)`

- Returns the character at the specified index.
- **Syntax**:

```
char charAt(int index);
```

- **Example**:

```
String str = "Hello";
System.out.println(str.charAt(1)); // Output: 'e'
```

3. `substring(int beginIndex)`

- Returns a new string that is a substring starting from the specified index.
- **Syntax**:

```
String substring(int beginIndex);
```

- **Example**:

```
String str = "Hello";
System.out.println(str.substring(2)); // Output: "llo"
```

4. `substring(int beginIndex, int endIndex)`

- Returns a substring from `beginIndex` to `endIndex-1`.
- **Syntax**:

```
String substring(int beginIndex, int endIndex);
```

- **Example**:

```
String str = "Hello";
System.out.println(str.substring(1, 4)); // Output: "ell"
```

5. `toUpperCase()`

- Converts all the characters of the string to uppercase.
- **Syntax**:

```
String toUpperCase();
```

- **Example**:

```
String str = "hello";
```

```
System.out.println(str.toUpperCase()); // Output: "HELLO"
```

6. `toLowerCase()`

- Converts all the characters of the string to lowercase.
- **Syntax**:

```
String toLowerCase();
```

- **Example**:

```
String str = "HELLO";
System.out.println(str.toLowerCase()); // Output: "hello"
```

7. `equals(Object obj)`

- Compares two strings for equality. Returns `true` if they are equal, otherwise `false`.
- **Syntax**:

```
boolean equals(Object obj);
```

- **Example**:

```
String str1 = "hello";
String str2 = "hello";
System.out.println(str1.equals(str2)); // Output: true
```

8. `equalsIgnoreCase(String anotherString)`

- Compares two strings for equality, ignoring case differences.
- **Syntax**:

```
boolean equalsIgnoreCase(String anotherString);
```

- **Example**:

```
String str1 = "HELLO";
String str2 = "hello";
System.out.println(str1.equalsIgnoreCase(str2)); // Output: true
```

9. `compareTo(String anotherString)`

- Compares two strings lexicographically (based on Unicode values). Returns a negative, zero, or positive value depending on whether the first string is lexicographically less than, equal to, or greater than the second string.
- **Syntax**:

```
int compareTo(String anotherString);
```

- **Example**:

```
String str1 = "apple";
String str2 = "banana";
System.out.println(str1.compareTo(str2)); // Output: -1 (because
"apple" is lexicographically less than "banana")
```

10. `contains(CharSequence sequence)`

- Checks if the string contains the specified sequence of characters.
- **Syntax**:

```
boolean contains(CharSequence sequence);
```

- **Example**:

```
String str = "Hello, World!";
System.out.println(str.contains("World")); // Output: true
```

11. `startsWith(String prefix)`

- Checks if the string starts with the specified prefix.
- **Syntax**:

```
boolean startsWith(String prefix);
```

- **Example**:

```
String str = "Hello, World!";
System.out.println(str.startsWith("Hello")); // Output: true
```

12. `endsWith(String suffix)`

- Checks if the string ends with the specified suffix.
- **Syntax**:

```
boolean endsWith(String suffix);
```

- **Example**:

```
String str = "Hello, World!";
System.out.println(str.endsWith("!")); // Output: true
```

13. `replace(char oldChar, char newChar)`

- Replaces all occurrences of the specified old character with the new character.
- **Syntax**:

```
String replace(char oldChar, char newChar);
```

- **Example**:

```
String str = "Hello, World!";
System.out.println(str.replace('o', '0')); // Output: "Hell0, W0rld!"
```

14. `trim()`

- Removes leading and trailing spaces from the string.
- **Syntax**:

```
String trim();
```

- **Example**:

```
String str = "  Hello, World!  ";
System.out.println(str.trim()); // Output: "Hello, World!"
```

15. `valueOf(Object obj)`

- Returns the string representation of the given object.
- **Syntax**:

```
String valueOf(Object obj);
```

- **Example**:

```
int num = 10;
System.out.println(String.valueOf(num)); // Output: "10"
```

16. `indexOf(String str)`

- Returns the index of the first occurrence of the specified substring.
- **Syntax**:

```
int indexOf(String str);
```

- **Example**:

```
String str = "Hello, World!";
System.out.println(str.indexOf("World")); // Output: 7
```

17. `lastIndexOf(String str)`

- Returns the index of the last occurrence of the specified substring.
- **Syntax**:

```
int lastIndexOf(String str);
```

- **Example**:

```
String str = "Hello, World! World!";
System.out.println(str.lastIndexOf("World")); // Output: 14
```
